arXiv:2408.02555v3 [cs.CV] 1 Dec 2024

MeshAnything V2: Artist-Created Mesh Generation with Adjacent Mesh Tokenization

Yiwen Chen[1,2] **Yikai Wang**[3*] **Yihao Luo**[4] **Zhengyi Wang**[2,3]
Zilong Chen[2,3] **Jun Zhu**[2,3] **Chi Zhang**[5*] **Guosheng Lin**[1*]

[1]Nanyang Technological University [2]Shengshu
[3]Tsinghua University [4]Imperial College London [5]Westlake University
https://buaacyw.github.io/meshanything-v2/

Abstract

Meshes are the de facto 3D representation in the industry but are labor-intensive to produce. Recently, a line of research has focused on autoregressively generating meshes. This approach processes meshes into a sequence composed of vertices and then generates them vertex by vertex, similar to how a language model generates text. These methods have achieved some success but still struggle to generate complex meshes. One primary reason for this limitation is their inefficient tokenization methods. To address this issue, we introduce MeshAnything V2, an advanced mesh generation model designed to create Artist-Created Meshes that align precisely with specified shapes. A key innovation behind MeshAnything V2 is our novel Adjacent Mesh Tokenization (AMT) method. Unlike traditional approaches that represent each face using three vertices, AMT optimizes this by employing a single vertex wherever feasible, effectively reducing the token sequence length by about half on average. This not only streamlines the tokenization process but also results in more compact and well-structured sequences, enhancing the efficiency of mesh generation. With these improvements, MeshAnything V2 effectively doubles the face limit compared to previous models, delivering superior performance without increasing computational costs. We will make our code and models publicly available.

1. Introduction

Due to the controllable and compact advantages of meshes, they serve as the predominant 3D representation in various industries, including games, movies, and virtual reality. For decades, the 3D industry has relied on human artists to manually create meshes, a process that is both time-consuming and labor-intensive.

To address this issue, very recently, a line of work [2, 4, 5, 15, 19, 27] has focused on automatically generating Artist-Created Meshes (AMs) to replace manual labor. Inspired by the success of large language models (LLMs), these approaches treat AMs as sequences of faces and learn to generate them with autoregressive transformers [23] in a manner similar to LLMs. Unlike methods that produce dense meshes in a reconstruction manner, these methods learn from the distribution of meshes created by human artists, thereby generating AMs that are efficient, beautiful, and can seamlessly replace manually created meshes.

Although these methods have achieved some success, they still face significant challenges. One major limitation is that current methods [2, 4, 5, 15, 19, 27] cannot generate meshes with a large number of faces. A primary reason for this constraint is the inadequacy of existing mesh tokenization methods. These methods treat a mesh as a sequence of faces, where each face consists of three vertices, and each vertex typically requires three tokens to represent. Consequently, each mesh is tokenized into a sequence nine times the number of its faces, resulting in substantial computational and memory demands. Besides, the resulting token sequence is highly redundant, which harms sequence learning and reduces performance.

Observing the above issues, we focus on the tokenization method of mesh generation in this work, aiming to improve the efficiency and quality of mesh generation. Extensive research in language models has demonstrated the importance of tokenization in sequence learning [10, 17, 29]. Moreover, unlike text, which inherently has a sequential structure, meshes are graph-based structures with 3D characteristics. For any given mesh, there are countless ways to represent it as a 1D token sequence, making the influence of tokenization methods even more pronounced for meshes. Therefore, we emphasize that research on mesh tokenization methods is of great importance.

* Corresponding authors.

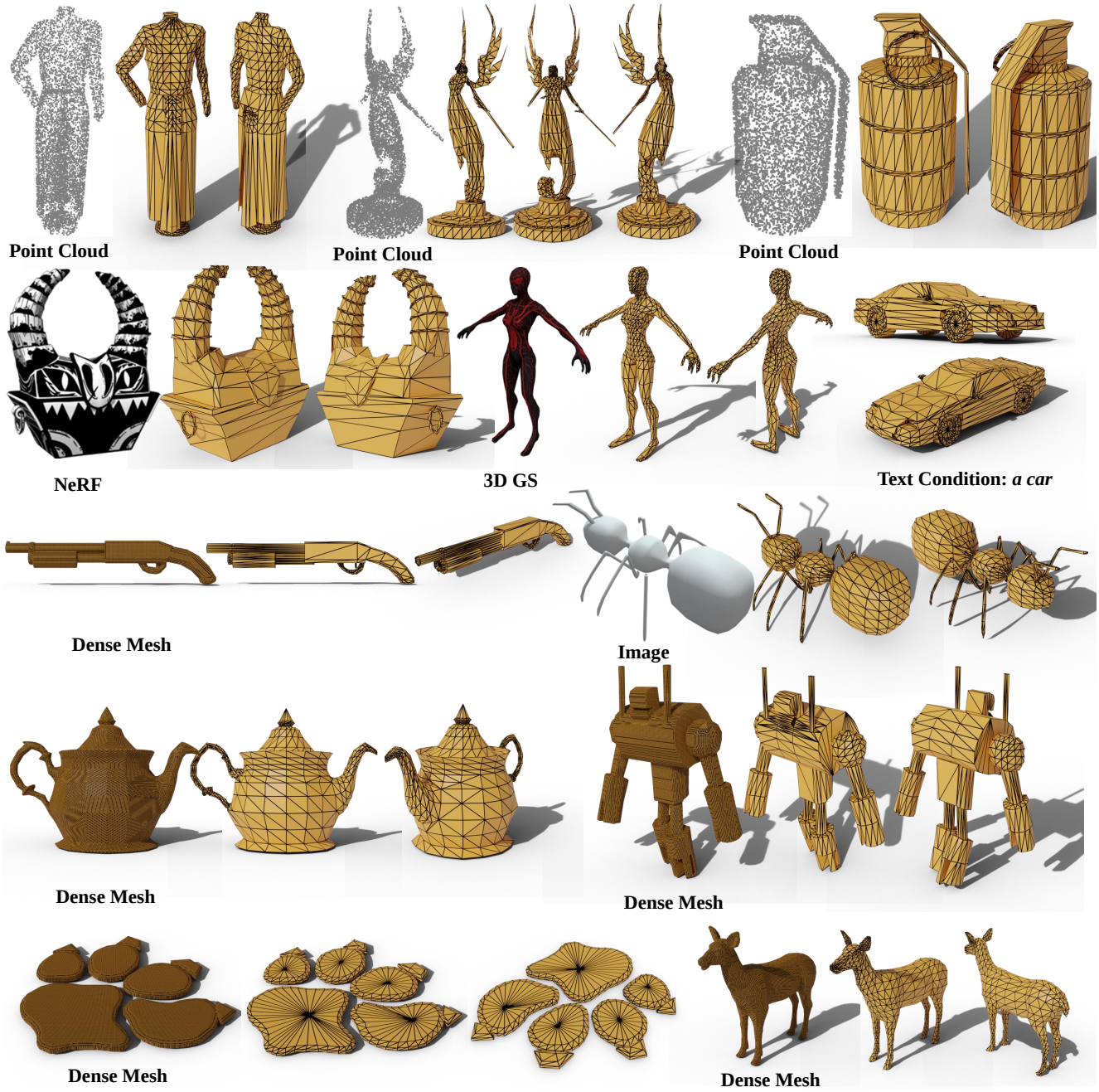

Figure 1. **Equipped with the newly proposed Adjacent Mesh Tokenization (AMT), MeshAnything V2 significantly surpasses MeshAnything [5] in both performance and efficiency.** MeshAnything V2 generates Artist-Created Meshes (AM) up to 1600 faces aligned with given shapes. Combined with various 3D asset production pipelines, it efficiently achieves high-quality, highly controllable AM generation.

The impact of mesh tokenization on autoregressive mesh generation can be considered in two main aspects. The first is efficiency: representing a mesh with shorter, more compact token sequences leads to reduced context length, thereby decreasing memory and computational complexity. The second aspect is the regularity of the token sequence. A shorter token sequence is not always better for mesh generation; the regularity and pattern consistency of the sequence are crucial for effective sequence learning. Effective mesh tokenization must balance both efficiency and regularity to achieve high-quality, efficient mesh generation.

Considering the above, we introduce MeshAnything V2, an advanced model for generating Artist-Created Meshes. This model incorporate several key improvements to boost

both performance and efficiency. At the core of MeshAnything V2 is our novel Adjacent Mesh Tokenization (AMT) method. AMT optimizes the tokenization process by representing each face with a single vertex rather than the traditional three. As illustrated in Fig. 1 and Algo. 1, AMT encodes adjacent faces using just one additional vertex, largely reducing the sequence length. When an adjacent face cannot be identified, a special token '&' is introduced to indicate this interruption, allowing the model to resume from an unencoded face.

In previous mesh generation works, users were unable to control the number of faces generated by the model, often resulting in meshes that did not meet application requirements. To address this, we introduce a face count condition, allowing users to specify an approximate number of faces, ensuring that generated meshes align with desired specifications. Additionally, to enhance the robustness of AMT during inference, we incorporate Masking Invalid Predictions [15] to prevent the model from producing invalid tokens, such as generating another '&' token immediately after an '&' token.

Extensive experiments on the Objaverse [6] dataset demonstrate that AMT can halve the sequence length on average. This reduction translates to nearly a fourfold decrease in computational load and memory usage within the attention block. Furthermore, AMT remains effective across various mesh generation settings, including unconditional and conditional approaches [19], even when using VQ-VAE [22]. Finally, with the integration of AMT, MeshAnything V2 doubles the maximum number of faces generated compared to previous models, significantly boosting both performance and efficiency.

In summary, our contributions are:

1. We introduce MeshAnything V2, a novel Artist-Created Mesh Generation model. V2 doubles the maximum number of faces that can be generated while achieving significantly better accuracy and efficiency.
2. The core of V2 is our newly proposed mesh tokenization method, Adjacent Mesh Tokenization (AMT). Compared to previous tokenization methods, AMT requires approximately half the token sequence length to represent the same mesh, thereby fundamentally reducing the computational burden of Artist-Created Mesh generation.
3. We conduct extensive experiments on various mesh tokenization methods to explore the essential properties required for effective mesh tokenization. Our experiments demonstrate that AMT significantly improves the efficiency and performance of mesh generation, and that the token sequence produced by mesh tokenization methods must balance compactness and regularity.

Algorithm 1: Adjacent Mesh Tokenization (AMT)

Input: $\mathcal{M}$: a triangle mesh
Output: A token sequence Seq that represents $\mathcal{M}$

```
Sort the vertices of M in ascending order by their coordinates;
Sort the faces of M in ascending order by their vertex indices;
Initialize an empty token sequence Seq;
Initialize a face list UnvisitedFaces as the faces of M;
Remove the first face from UnvisitedFaces;
Append the three vertices of the removed face to Seq;
Define a SpecialToken & to indicate break signal.
while UnvisitedFaces is not empty do
    if the last token in Seq is & then
        Remove the first face from UnvisitedFaces;
        Append the three vertices of the removed face to Seq;
    else
        AdjacentVertices ← vertices adjacent to both the last two vertices in Seq;
        Filter out vertices from AdjacentVertices that form faces with the last two vertices in Seq that are not in UnvisitedFaces;
        if AdjacentVertices is not empty then
            Sort AdjacentVertices in ascending order by their coordinates;
            Append the first vertex in AdjacentVertices to Seq;
        else
            Append & to Seq;
return Seq;
```

2. Related Works

2.1. Artist-Created Mesh Generation

Diverging from previous works that produce dense meshes, recent works have focused on generating meshes created by human artists, i.e., Artist-Created Meshes (AMs) [2, 4, 5, 15, 19, 27]. These methods process meshes into ordered face sequences and learn to generate this sequence. [15] first proposed using autoregressive transformers to sequentially generate vertices and faces. [19] use VQ-VAE to learn a mesh vocabulary and then learn this vocabulary with a decoder-only transformer. [2] differ from other methods by using a discrete diffusion model to generate AMs instead of an autoregressive transformer. [4] propose directly using the discretized coordinates of the vertex as the token index,

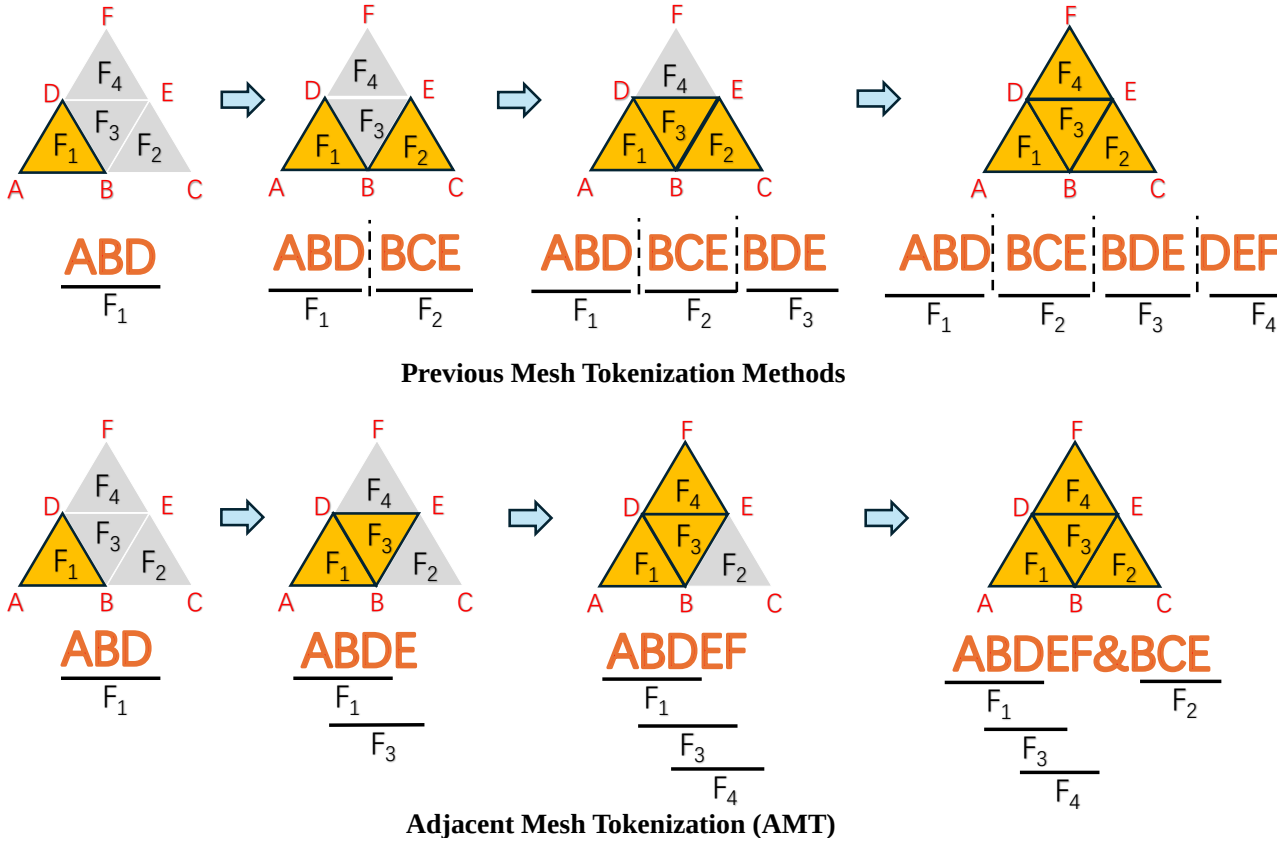

Figure 2. **Illustration of Adjacent Mesh Tokenization (AMT).** Unlike previous methods that use three vertices to represent a face, AMT uses a single vertex whenever possible. When this is impossible, AMT adds a special token & and restarts. Our experiments demonstrate that AMT reduces the token sequence length by half on average. Its compact, and well-structured sequence representation enhances sequence learning, thereby significantly improving both the efficiency and performance of mesh generation.

bypassing the need for VQ-VAE as in [19]. [27] use pivot vertices as a coarse mesh representation and then generate the complete mesh tokens. [5] generate AMs aligned with given shapes, which can be integrated with various 3D asset production methods to convert their results into AMs.

As shown in Fig. 2, all of these methods process meshes into face sequences and use three vertices to represent a single face, resulting in highly redundant representations. Different from these methods, our newly proposed Adjacent Mesh Tokenization (AMT) uses a single vertex to represent a single face, providing a more compact and well-structured mesh representation, thereby significantly improving the efficiency and performance of mesh generation.

2.2. 3D Generation

In recent years, 3D generation has gradually become one of the mainstream research directions in the field of 3D research. This area focuses on generating diverse, high-quality 3D assets for the 3D industry. Generative Adversarial Networks (GANs) [1, 7, 28] produce synthetic 3D data by training a generator and a discriminator network to distinguish between generated and real data. Very recently, a new line of works [9, 11, 12, 18, 20, 21, 25, 26, 30] directly generate 3D assets in a feed-forward manner. [9] pioneer these methods and use a transformer to directly regress the parameters of 3D models given conditions. Besides, applying diffusion models [8] to directly generate 3D assets has also been widely researched [2, 13, 14, 16, 31, 34]. [31] lead the SOTA of current 3D generation methods by first generating high-quality 3D shapes with DiT and then producing detailed textures with material diffusion models.

As mesh is a crucial component in 3D generation, Artist-Created Mesh Generation [2, 4, 5, 15, 19, 27] is closely related to 3D generation. However, it differs significantly from previous 3D generation methods as it mainly focuses on sequence learning, which is rarely seen in other 3D generation methods, to produce high-quality mesh topology.

3. Method

In this work, we introduce a novel mesh tokenization method, named Adjacent Mesh Tokenization (AMT). Compared to

previous tokenization methods, AMT reduces the token sequence length by approximately half to represent the same mesh, thereby significantly lowering the computational overhead of mesh generation. AMT is detailed in Section 3.1.

Next, in Section 3.2, we integrate AMT into MeshAnything [5] and introduce MeshAnything V2.

3.1. Adjacent Mesh Tokenization

In this section, we detail Adjacent Mesh Tokenization (AMT), a novel tokenization method for Artist-Created Mesh (AM) generation. Compared to previous methods, AMT processes the mesh into a more compact and well-structured token sequence by representing each face with a single vertex whenever possible. For simplicity, we describe AMT on triangle mesh. But it is worth noting that AMT can be easily generalized to the generation of meshes with variable polygons.

Tokenization is a crucial part of sequence learning, as it processes various data formats, such as text, images, and audio, into token sequences. The processed tokens are then used as ground truth inputs for training the sequence model. During inference, the sequence model generates a token sequence that is subsequently detokenized into the target data format. Therefore, tokenization plays a vital role in sequence learning, determining the quality of the data sequence that the sequence model learns from.

We first illustrate the tokenization methods used in previous methods [2, 4, 5, 15, 19, 27]. Although there are slight differences in detail, the previous tokenization methods can be unified as follows: Given a mesh $\mathcal{M}$, vertices are first sorted in ascending order based on their z-y-x coordinates, where z represents the vertical axis. Next, faces are ordered by their lowest vertex index, then by the next lowest, and so on. The mesh is then viewed as an **ordered** sequence of faces:

$$\mathcal{M} := (f_1, f_2, f_3, \ldots, f_N), \tag{1}$$

where f_i represents the i-th face in the mesh, and N is the number of faces in $\mathcal{M}$.

Then, each f_i is represented as an ordered sequence of three vertices v:

$$f_i := (v_{i1}, v_{i2}, v_{i3}), \tag{2}$$

where v_{i1}, v_{i2}, and v_{i3} are the vertices that form the i-th face f_i in the mesh. It is worth noting that v_{i1}, v_{i2}, and v_{i3} have already been sorted and have a fixed order.

Substituting Equation (2) into Equation (1) gives:

$$\mathcal{M} := ((v_{11}, v_{12}, v_{13}), \ldots, (v_{N1}, v_{N2}, v_{N3})) = \text{Seq}_V \tag{3}$$

Due to the sorting, the resulting Seq_V is unique and its length is three times the number of faces in the mesh. It is evident that Seq_V contains a significant amount of redundant information, as each vertex appears as many times as the number of faces it belongs to.

To resolve this issue, we propose Adjacent Mesh Tokenization (AMT) to obtain a more compact and well-structured Seq_V than previous method. Our key observation is that the main redundancy of Seq_V comes from representing each face with three vertices as in (2). This results in vertices that have already been visited appearing redundantly in Seq_V. Therefore, AMT aims to represent each face using only a single vertex whenever possible. As shown in Fig. 2 and Algo. 1, AMT efficiently encodes adjacent faces during tokenization, using only one additional vertex. When no adjacent face is available, as illustrated in the last step of Fig. 2, AMT inserts a special token & into the sequence to denote this event and restarts the process from a face that has not yet been encoded. To detokenize, simply reverse the tokenization algorithm as described in Algo. 1.

In the ideal case, where the special token "&" is rarely used, AMT can reduce the length of Seq_V obtained by previous methods to nearly one-third. Of course, in extreme cases, such as when each face in the mesh is completely disconnected from others, AMT performs worse than previous methods. However, since the datasets [3, 6] used for AM generation are created by human artists, the meshes generally have well-structured topologies. Thus, the overall performance of AMT is significantly better than previous methods. As shown in Sec. 4.3, on the Objaverse test set, AMT can reduce the length of Seq_V by half on average.

Vertices Swap. Considering two faces f_1 and f_2:

$$f_1 = (v_1, v_2, v_3), f_2 = (v_1, v_3, v_4).$$

These two faces are connected by edge (v_1, v_3). Assuming we first represent f_1 as (v_1, v_2, v_3), AMT will be interrupted because f_2 does not contain v_2, even though f_1 and f_2 are actually adjacent. To solve this issue, we introduce a special token \$ to swap vertices. When the \$ token appears in front of a vertex, it indicates that the next face will be formed by the first and the last vertex of the previous face, instead of the last two. For example, the token sequence $(v_1, v_2, v_3, \$, v_4)$ implies the mesh consists of two faces: (v_1, v_2, v_3) and (v_1, v_3, v_4), i.e. f_1 and f_2.

When swap is enabled, AMT can explore more adjacent faces when searching for the next face. Specifically, in line 13 of Algo. 1, $AdjacentVertices$ can be modified to additionally include vertices adjacent to the last and third-to-last vertices in Seq.

Although the swap operation introduces an additional special token, it reduces the number of interruptions and effectively shortens the token sequence. While adding a special token may potentially increase the difficulty of sequence learning, our experiments in Sec. 4.3 show that it has no noticeable impact.

Discussion on Sorting in Mesh Tokenization. Both previous methods and AMT initially sort the vertices and faces of the mesh. The primary goal is to process the mesh data into a sequence with a fixed pattern, making it easier for the learning of the sequence model. In AMT, to maintain this design, we consistently choose the face with the earlier index in the sorted list whenever there are multiple choices. Besides, thanks to this design, the token sequence processed by AMT is unique for each mesh. Additionally, AMT prioritizes visiting adjacent faces whenever possible. In contrast, previous methods naively follow the sorted order, often resulting in token sequences where spatially distant vertices are adjacent in the sequence, potentially increasing sequence complexity. As shown in Sec. 4.3, compared to previous methods, AMT demonstrates significant advantages in both speed and memory usage, as well as improved accuracy, proving that the sequences generated by AMT are more compact and well-structured.

Discussion on AMT and the use of VQ-VAE. After obtaining Seq_V, mesh generation methods then need to process it into a token sequence for sequence learning. [19] propose to train a VQ-VAE [22] to achieve this. They take Seq_V as input and learn a vocabulary of geometric embeddings with the VQ-VAE. After training the VQ-VAE, they then use the VQ-VAE's quantized features as the input for the transformer [23]. Very recently, [4] proposed another method to process Seq_V into a token sequence. They discard VQ-VAE and directly use the discretized coordinates of the vertex as the token index. It is important to emphasize that whether or not VQ-VAE is used does not affect AMT's effectiveness. This is because AMT operates before the aforementioned methods. For example, in the case of using VQ-VAE, AMT first shortens the Seq_V that represents $\mathcal{M}$, and the shortened Seq_V is then quantized into an embedding sequence with the VQ-VAE.

3.2. MeshAnything V2

In this section, we introduce MeshAnything V2. It is equipped with AMT and scales up its maximum generated face count from 800 to 1600. Without increasing the number of parameters, MeshAnything V2 achieves shape conditioned Artist-Created Mesh (AM) generation with significantly better performance and efficiency. We also use it as an example to demonstrate how AMT can be applied to mesh generation.

Following [5], MeshAnything V2 also targets generating AMs aligned to a given shape, allowing integration with various 3D asset production pipelines to achieve highly controllable AM generation. That is, we aim to learn the distribution: $p(\mathcal{M}|\mathcal{S})$, where $\mathcal{M}$ represents the AM and $\mathcal{S}$ represents the 3D shape condition.

As in [5], V2 uses point clouds as the shape condition input $\mathcal{S}$. We also use the same point cloud–Artist-Created Mesh data pairs $(\mathcal{M}, \mathcal{S})$ collected in [5]. The target distribution $p(\mathcal{M}|\mathcal{S})$ is learned with a decoder-only transformer with the same size and architecture as in [5]. To inject $\mathcal{S}$ into the transformer, we first encode it with a pretrained point cloud encoder [33] into a fixed-length token sequence $\mathcal{T}_S$ and then set it as the prefix of the transformer's token sequence. We then process paired $\mathcal{M}$ into mesh token sequence $\mathcal{T}_M$. It is concatenated to the point cloud token sequence as the transformer's ground truth sequence. After training the transformer with cross-entropy loss, we input $\mathcal{T}_S$ and let the transformer autoregressively generate the corresponding $\mathcal{T}_M$, which is then detokenized into $\mathcal{M}$.

The key difference between [5] and our method is the way we obtain $\mathcal{T}_M$. Instead of the naive mesh tokenization method used in [5], we process $\mathcal{M}$ with the newly proposed Adjacent Mesh Tokenization (AMT) and obtain a more compact and efficient sequence Seq_V. Following [4], we discard the VQ-VAE and directly use the discretized coordinates from Seq_V as token indices. We then add a newly initialized codebook entry to represent the & in the AMT sequence. Finally, we sequentially combine the coordinate token sequence and the special token for & to obtain the mesh token sequence $\mathcal{T}_M$ for transformer input. As mentioned in Sec. 3.1, it is worth noting that whether or not VQ-VAE is used does not affect the application and effectiveness of AMT.

To facilitate the transformer's learning of the sequence patterns of AMT, in addition to the absolute positional encoding used in [5], we add the following embeddings for Adjacent Mesh Tokenization (AMT): when representing a face with three vertices, we add a specific embedding for the three new vertices; when representing a face with one vertex, we add a different embedding for the single new vertex. Additionally, we provide a distinct embedding for the & token.

Face Count Condition. Considering that some applications require control over the approximate face count, we explored the addition of a face count condition in mesh generation. We initialized an embedding book with a size equal to the maximum allowed number of faces. Based on the current face count in the mesh, we retrieve the corresponding embedding from this book and place it after the point cloud prefix to represent the face count. During training, we add a random variable to the face count to introduce some variation and prevent overfitting to an exact condition. During training, we add a random variable to the face count to introduce some variation and prevent overfitting to an exact condition. Additionally, we drop this condition with a 10% probability to further improve condition robustness.

Masking Invalid Predictions. PolyGen [15] introduces Masking Invalid Predictions during inference. In PolyGen, vertices are generated first, followed by faces, and both vertices and faces are sorted. Without applying constraints dur-

Table 1. **Ablation Study on AMT.** We compare MeshAnything V2 with its variant without AMT. Please refer to Section 4.3 for detailed explanation.

Method	CD↓ ($\times 10^{-2}$)	ECD↓ ($\times 10^{-2}$)	NC↑	#V↓	#F↓	V_Ratio↓	F_Ratio↓	S_Ratio↓
V2 W/O AMT	0.895	4.832	0.924	**302.4**	**556.7**	**1.105**	**1.062**	1.000
V2	**0.874**	**4.721**	**0.933**	308.6	571.8	1.127	1.097	**0.497**

ing inference, generative models may produce results that do not follow the sequence structure, such as generating faces before completing all vertices or generating vertices that violate the sorted coordinates order. PolyGen [15] addresses this issue by masking invalid logits during the inference phase, ensuring that only valid results are generated.

We followed this design in our PolyGen experiments. Due to the widespread use of coordinate sorting in meshes, this design can also be adopted in other tokenization methods. In the AMT experiments, we incorporated this design as well. For example, we enforced a rule in AMT where, when starting a new strip, at least three vertices must be generated before allowing any interruptions.

4. Experiments

4.1. Implementation Details

The main experimental setting of MeshAnything V2 remains consistent with [5], except we equipped it with the newly proposed Adjacent Mesh Tokenization (AMT). We still use OPT-350M [32] as our autoregressive transformer [23] and the pretrained point encoder from [33].

For dataset preparation, we adopt a similar dataset preparation technique as used in [5], with several modifications. Firstly, we noticed that Objaverse contains some duplicate mesh data, where the meshes are identical in shape but differ in texture. We detected these duplicates by calculating the differences in vertex coordinates and retained only one instance of each mesh. Additionally, in [5], the ground truth meshes were first processed into SDF using mesh2sdf [24]. While mesh2sdf occasionally produces some failure cases, we filtered out these failures using Chamfer Distance. Different from the dataset used in [5], we used meshes with fewer than 1600 faces as the experimental dataset, compared to 800 in the previous work, and also included a small portion of data from ObjaverseXL. This resulted in a dataset containing 230K point clouds and mesh pairs. We randomly sampled 4K data samples as the evaluation dataset.

To accommodate meshes with more faces, we sampled 8192 points instead of 4096 for each point cloud. Besides, unlike [5], we update the point encoder from [33] during training because we find its accuracy insufficient for handling complex meshes with up to 1600 faces.

MeshAnything V2 is trained with 32 A800 GPUs for four days. The batch size per GPU is 8, resulting in a total batch size of 256.

4.2. Qualitative Experiments

We present the qualitative results of MeshAnything V2. As shown in Fig. 1, MeshAnything V2 effectively generates high quality Artist-Created Mesh aligned to given shapes. When integrated with various 3D assets production pipelines, V2 successfully achieves highly contrabllable AM generation.

4.3. Quantitative Experiments

Evaluation Metrics. We follow the evaluation metric settings of [5] to quantitatively assess mesh quality. We uniformly sample point clouds from both the ground truth meshes and the generated meshes and compute the following metrics on our 2K evaluation dataset:

- **Chamfer Distance (CD):** Evaluates the overall quality of a reconstructed mesh by computing chamfer distance between point clouds.
- **Edge Chamfer Distance (ECD):** Assesses the preservation of sharp edges by sampling points near sharp edges and corners.
- **Normal Consistency (NC):** Evaluates the quality of the surface normals.
- **Number of Mesh Vertices (#V):** Counts the vertices in the mesh.
- **Number of Mesh Faces (#F):** Counts the faces in the mesh.
- **Vertex Ratio (V_Ratio):** The ratio of the estimated number of vertices to the ground truth number of vertices.
- **Face Ratio (F_Ratio):** The ratio of the estimated number of faces to the ground truth number of faces.
- **Perplexity:** We report training perplexity to measure how well the mesh generation model predicts a sample. A lower perplexity indicates more accurate predictions.
- **Sequence Ratio (S_Ratio):** The sequence length compression ratio of the used mesh tokenization methods.

Ablation Study. We ablate the effectiveness of AMT by comparing the results of MeshAnything V2 with its variant without AMT. The variant follows exactly the same settings as V2, except that AMT is replaced with naive mesh tokenization as in previous methods [4, 5, 19]. This also serves as a fairer comparison with [5], as the original model of [5] was trained on data with fewer than 800 faces rather than 1600. Moreover, [5] is trained on a small total batch size of 64 while we observed a noticeable performance improvement when the batch size was increased to V2's 256. We

Table 2. **Comparison of Tokenization Methods.** We compare various mesh tokenization methods and their impact on the generated mesh results in mesh generation. Note that the metrics are calculated by sampling 10K points cloud on each mesh.

Method	CD↓ ($\times 10^{-2}$)	ECD↓ ($\times 10^{-2}$)	NC↑	#V↓	#F↓	V_Ratio↓	F_Ratio↓	S_Ratio↓	Perplexity↓
Baseline	2.478	**18.21**	0.893	95.4	178.9	0.974	0.940	1.000	**1.150**
Unsort	8.151	31.86	0.794	117.4	213.1	1.189	1.219	1.000	1.234
PolyGen(AMT)	3.226	22.97	0.872	**93.2**	**172.2**	**0.928**	**0.936**	**0.372**	1.589
AMT	**2.348**	19.33	0.904	102.7	187.1	1.013	0.983	0.492	1.363
AMT(Swap)	2.517	19.86	**0.913**	110.8	198.9	1.098	1.102	0.455	1.416

sample 100K points on each mesh to calculate the aforementioned metrics.

As shown in Table 1, our results indicate that V2 significantly outperforms its variant, demonstrating the effectiveness of AMT. It shows that AMT not only improves training speed and reduces memory pressure but also enhances generation quality. Notably, although V2 and its variant were trained for the same number of iterations, the variant consumed nearly two times the GPU hours of V2. Additionally, the table shows that AMT takes slightly more vertices and faces. A likely reason for this is that the variant's performance is still too weak, and it tends to ignore details and use a simpler topology when representing complex meshes with high face counts. We also observe that both AMT and the variant have vertex and face ratios greater than 1.0, meaning they use more faces on average relative to the ground truth, unlike the results in [5], which were less than 1 (around 0.88). We suspect this is because the dataset for V2 contains more complex meshes with over 800 faces, which causes the model to occasionally produce more complex topology for simple shapes.

Comparison of Tokenization Methods. Below, we list the methods to be compared in the following sections, along with their specific implementation details. In the tokenization methods comparison experiments, we use the smaller OPT-125M [32] and select meshes with up to 400 faces from the aforementioned datasets as the experimental dataset. We sample 10K points on each mesh to calculate the aforementioned metrics.

- **Baseline.** This refers to the naive mesh tokenization method used in previous works, where each face is strictly represented by three vertices.
- **PolyGen(AMT).** This is a combination of the tokenization method from PolyGen [15] with AMT. Polygen first generates vertex coordinates and then expresses the faces by generating vertex indices. We discuss this setting in detail in the Supplementary Materials.
- **AMT.** This refers to the Adjacent Mesh Tokenization proposed in Sec. 3.1, as shown in Figure Aug1.
- **AMT(Swap).** As described in Sec. 3.1, this method combines the AMT algorithm with vertices swap. An additional special token is used to represent the swap operation.
- **Unsort.** This is based on the naive mesh tokenization method from previous works [15, 19], but without sorting the mesh. Instead, the mesh is used in the order in which it is stored. It is important to note that the meshes in Objaverse [6] are created by human, and their storage may already include some level of inherent order. The purpose of this method is to compare with the **Baseline** and verify the importance of sorting.

The results are shown in Tab. 2. Unsort vs Baseline comparison validates the importance of tokenization method regularity for mesh generation.

PolyGen(AMT) [15] achieves the best S_Ratio (sequence length compression ratio), indicating its high efficiency. However, there is a noticeable performance gap compared to Baseline and AMT, indicating the produced mesh quality is not high enough. The primary reason for this might be that the autoregressive model struggles to interact accurately with previously generated vertices when producing faces, which increases the difficulty of sequence learning.

The results from AMT demonstrates that it shortens the token length without compromising mesh quality, as confirmed by its comparison with the Baseline. Additionally, in AMT(Swap), the introduction of the Swap special token further increases the compression ratio without affecting mesh quality.

From the above experiments, we can conclude that both the regularity of the token sequence and the compression ratio are crucial. Although PolyGen [15] achieves a better compression ratio, its token sequence organization is not well-suited for sequence learning, making it a sub-optimal mesh tokenization method compared to AMT. In contrast, both AMT and AMT(Swap) reduce the compression ratio without increasing the difficulty of sequence learning, making them more suitable for mesh generation.

5. Conclusion

In this work, we present MeshAnything V2, a shape-conditioned Artist-Created Mesh (AM) generation model that generates AM aligned to given shapes. V2 significantly outperfroms MeshAnything [5] in both performance and efficiency with our newly proposed Adjacent Mesh Tokenization (AMT). Different from previous methods that use three ver-

tices to represent a face, AMT uses a single vertex whenever possible. Our experiments demonstrate that AMT averagely reduces the token sequence length by half. The compact, and well-structured token sequence from AMT greatly enhances sequence learning, thereby significantly improving the efficiency and performance of AM generation.

References

[1] Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas Guibas. Learning representations and generative models for 3d point clouds. In *International conference on machine learning*, pages 40–49. PMLR, 2018. 4

[2] Antonio Alliegro, Yawar Siddiqui, Tatiana Tommasi, and Matthias Nießner. Polydiff: Generating 3d polygonal meshes with diffusion models. *arXiv preprint arXiv:2312.11417*, 2023. 1, 3, 4, 5

[3] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. Shapenet: An information-rich 3d model repository. *arXiv preprint arXiv:1512.03012*, 2015. 5

[4] Sijin Chen, Xin Chen, Anqi Pang, Xianfang Zeng, Wei Cheng, Yijun Fu, Fukun Yin, Yanru Wang, Zhibin Wang, Chi Zhang, et al. Meshxl: Neural coordinate field for generative 3d foundation models. *arXiv preprint arXiv:2405.20853*, 2024. 1, 3, 4, 5, 6, 7

[5] Yiwen Chen, Tong He, Di Huang, Weicai Ye, Sijin Chen, Jiaxiang Tang, Xin Chen, Zhongang Cai, Lei Yang, Gang Yu, Guosheng Lin, and Chi Zhang. Meshanything: Artist-created mesh generation with autoregressive transformers, 2024. 1, 2, 3, 4, 5, 6, 7, 8, 11

[6] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *CVPR*, pages 13142–13153, 2023. 3, 5, 8

[7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020. 4

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 4

[9] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. Lrm: Large reconstruction model for single image to 3d. *arXiv preprint arXiv:2311.04400*, 2023. 4

[10] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018. 1

[11] Jiahao Li, Hao Tan, Kai Zhang, Zexiang Xu, Fujun Luan, Yinghao Xu, Yicong Hong, Kalyan Sunkavalli, Greg Shakhnarovich, and Sai Bi. Instant3d: Fast text-to-3d with sparse-view generation and large reconstruction model. *arXiv preprint arXiv:2311.06214*, 2023. 4

[12] Minghua Liu, Chao Xu, Haian Jin, Linghao Chen, Mukund Varma T, Zexiang Xu, and Hao Su. One-2-3-45: Any single image to 3d mesh in 45 seconds without per-shape optimization. *Advances in Neural Information Processing Systems*, 36, 2024. 4

[13] Zhen Liu, Yao Feng, Michael J Black, Derek Nowrouzezahrai, Liam Paull, and Weiyang Liu. Meshdiffusion: Score-based generative 3d mesh modeling. *arXiv preprint arXiv:2303.08133*, 2023. 4

[14] Zhaoyang Lyu, Jinyi Wang, Yuwei An, Ya Zhang, Dahua Lin, and Bo Dai. Controllable mesh generation through sparse latent point diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 271–280, 2023. 4

[15] Charlie Nash, Yaroslav Ganin, SM Ali Eslami, and Peter Battaglia. Polygen: An autoregressive generative model of 3d meshes. In *International conference on machine learning*, pages 7220–7229. PMLR, 2020. 1, 3, 4, 5, 6, 7, 8, 11

[16] Alex Nichol, Heewoo Jun, Prafulla Dhariwal, Pamela Mishkin, and Mark Chen. Point-e: A system for generating 3d point clouds from complex prompts. *arXiv preprint arXiv:2212.08751*, 2022. 4

[17] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, 2016. 1

[18] Yichun Shi, Peng Wang, Jianglong Ye, Mai Long, Kejie Li, and Xiao Yang. Mvdream: Multi-view diffusion for 3d generation. *arXiv preprint arXiv:2308.16512*, 2023. 4

[19] Yawar Siddiqui, Antonio Alliegro, Alexey Artemov, Tatiana Tommasi, Daniele Sirigatti, Vladislav Rosov, Angela Dai, and Matthias Nießner. Meshgpt: Generating triangle meshes with decoder-only transformers. *arXiv preprint arXiv:2311.15475*, 2023. 1, 3, 4, 5, 6, 7, 8, 11

[20] Jiaxiang Tang, Zhaoxi Chen, Xiaokang Chen, Tengfei Wang, Gang Zeng, and Ziwei Liu. Lgm: Large multi-view gaussian model for high-resolution 3d content creation. *arXiv preprint arXiv:2402.05054*, 2024. 4

[21] Dmitry Tochilkin, David Pankratz, Zexiang Liu, Zixuan Huang, Adam Letts, Yangguang Li, Ding Liang, Christian Laforte, Varun Jampani, and Yan-Pei Cao. Triposr: Fast 3d object reconstruction from a single image. *arXiv preprint arXiv:2403.02151*, 2024. 4

[22] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017. 3, 6

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 6, 7

[24] Peng-Shuai Wang, Yang Liu, and Xin Tong. Dual octree graph networks for learning adaptive volumetric shape representations. *ACM Transactions on Graphics (TOG)*, 41(4): 1–15, 2022. 7

[25] Zhengyi Wang, Yikai Wang, Yifei Chen, Chendong Xiang, Shuo Chen, Dajiang Yu, Chongxuan Li, Hang Su,

and Jun Zhu. Crm: Single image to 3d textured mesh with convolutional reconstruction model. *arXiv preprint arXiv:2403.05034*, 2024. 4

[26] Xinyue Wei, Kai Zhang, Sai Bi, Hao Tan, Fujun Luan, Valentin Deschaintre, Kalyan Sunkavalli, Hao Su, and Zexiang Xu. Meshlrm: Large reconstruction model for high-quality mesh. *arXiv preprint arXiv:2404.12385*, 2024. 4

[27] Haohan Weng, Yikai Wang, Tong Zhang, CL Chen, and Jun Zhu. Pivotmesh: Generic 3d mesh generation via pivot vertices guidance. *arXiv preprint arXiv:2405.16890*, 2024. 1, 3, 4, 5

[28] Jiajun Wu, Chengkai Zhang, Tianfan Xue, Bill Freeman, and Josh Tenenbaum. Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling. *Advances in neural information processing systems*, 29, 2016. 4

[29] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. In *arXiv preprint arXiv:1609.08144*, 2016. 1

[30] Jiale Xu, Weihao Cheng, Yiming Gao, Xintao Wang, Shenghua Gao, and Ying Shan. Instantmesh: Efficient 3d mesh generation from a single image with sparse-view large reconstruction models. *arXiv preprint arXiv:2404.07191*, 2024. 4

[31] Longwen Zhang, Ziyu Wang, Qixuan Zhang, Qiwei Qiu, Anqi Pang, Haoran Jiang, Wei Yang, Lan Xu, and Jingyi Yu. Clay: A controllable large-scale generative model for creating high-quality 3d assets. *ACM Transactions on Graphics (TOG)*, 43 (4):1–20, 2024. 4

[32] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022. 7, 8

[33] Zibo Zhao, Wen Liu, Xin Chen, Xianfang Zeng, Rui Wang, Pei Cheng, Bin Fu, Tao Chen, Gang Yu, and Shenghua Gao. Michelangelo: Conditional 3d shape generation based on shape-image-text aligned latent representation. *Advances in Neural Information Processing Systems*, 36, 2024. 6, 7

[34] Linqi Zhou, Yilun Du, and Jiajun Wu. 3d shape generation and completion through point-voxel diffusion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5826–5835, 2021. 4

A. Additional Experiments

We provide additional quantitative experiments and detailed discussion about Polygen [15] tokenization method here.

A.1. Experiments on Face Count Condition.

We tested the effectiveness of the face count condition by scaling the ground truth face count value by a scalar. We used the V2 model trained on the mesh dataset with fewer than 1600 faces as the test model and sampled 10K point clouds for each mesh to calculate the metrics. During inference, for each ground truth mesh, we input the scaled face count and the paired point cloud into the model and measured the effect on face count and mesh quality. As shown in Tab. 3, at a scale ratio of 0.8, the model significantly reduced the face count while maintaining mesh quality. However, at a scale ratio of 0.6, the face count did not decrease further, indicating that while the model has some ability to follow the face condition, it prioritizes mesh quality when the condition becomes difficult to meet. Similarly, when the scale ratio is set to greater than 1, the model exhibits similar behavior.

A.2. User Study.

We conducted a user study on mesh generation quality to compare MeshAnything [5] and our works. For a fair comparison, we randomly sampled 30 meshes with fewer than 800 faces from the evaluation dataset and input their paired point clouds into each model. Users were asked to select the result they preferred from the two options. We collected responses from a total of 43 users, and the voting rates for [5] and MeshAnything V2 were 32.2% and 67.8%, respectively. This indicates that the results generated by V2 are more aligned with human preference.

A.3. Discussion on PolyGen Tokenization Method.

Polygen[15] introduces a mesh tokenization approach that differs from other existing methods [19] for mesh generation. It first employs an autoregressive vertex model to generate the 3D coordinates of the mesh's vertices. These vertices are then used as a prefix and fed into another autoregressive face model, which connects these vertices into faces, thus constructing the entire mesh. Since the 3D coordinates are already provided by the vertex model, the face model does not need to estimate the 3D coordinates, but only specifies the connections using vertex indices, significantly reducing the sequence length. During the face generation, this tokenization method can be combined with AMT further shorten the token sequence length.

By using vertex indices to represent faces, PolyGen [15] tokenization consumes only one token to define a face, whereas other methods require three tokens to represent a single vertex. Assuming a vertex is referenced n times, PolyGen's tokenization requires $3 + n$ tokens, whereas other tokenization methods would require $3 \times n$ tokens. Although PolyGen spends one additional token when a vertex is referenced only once, in most cases, each vertex is referenced multiple times, allowing PolyGen's approach to save sequence length.

Although PolyGen tokenization effectively reduces the token sequence length, this generation method requires the model to accurately predict vertex positions first, which may not be ideal for an autoregressive model.

In the experiments section of our main paper, we compare Polygen tokenization method with other methods. To make a fair comparison, we merge the two stage generation process of PolyGen [15] into a single model that first generates vertex coordinates and then expresses the faces by generating vertex indices. Besides, we combine PolyGen's face generation stage with AMT to further reduce the token sequence length.

Table 3. **Experiments on Face Count Condition.** We control the face count condition using the scale ratio, where 1.0 indicates using the ground truth face count as the condition. The experiments show that our face count condition has the ability to control the number of faces.

Scale Ratio	CD↓ ($\times 10^{-2}$)	V_Ratio	F_Ratio
1.0	1.768	1.127	1.097
0.8	**1.734**	0.928	0.912
0.6	1.822	**0.902**	**0.882**
1.2	1.814	1.282	1.248
1.4	1.920	1.271	1.252

B. Limitations.

Although there is a large improvement over V1, the accuracy of MeshAnything V2 is still insufficient for industrial applications. More efforts are needed to improve the model's stability and accuracy.